

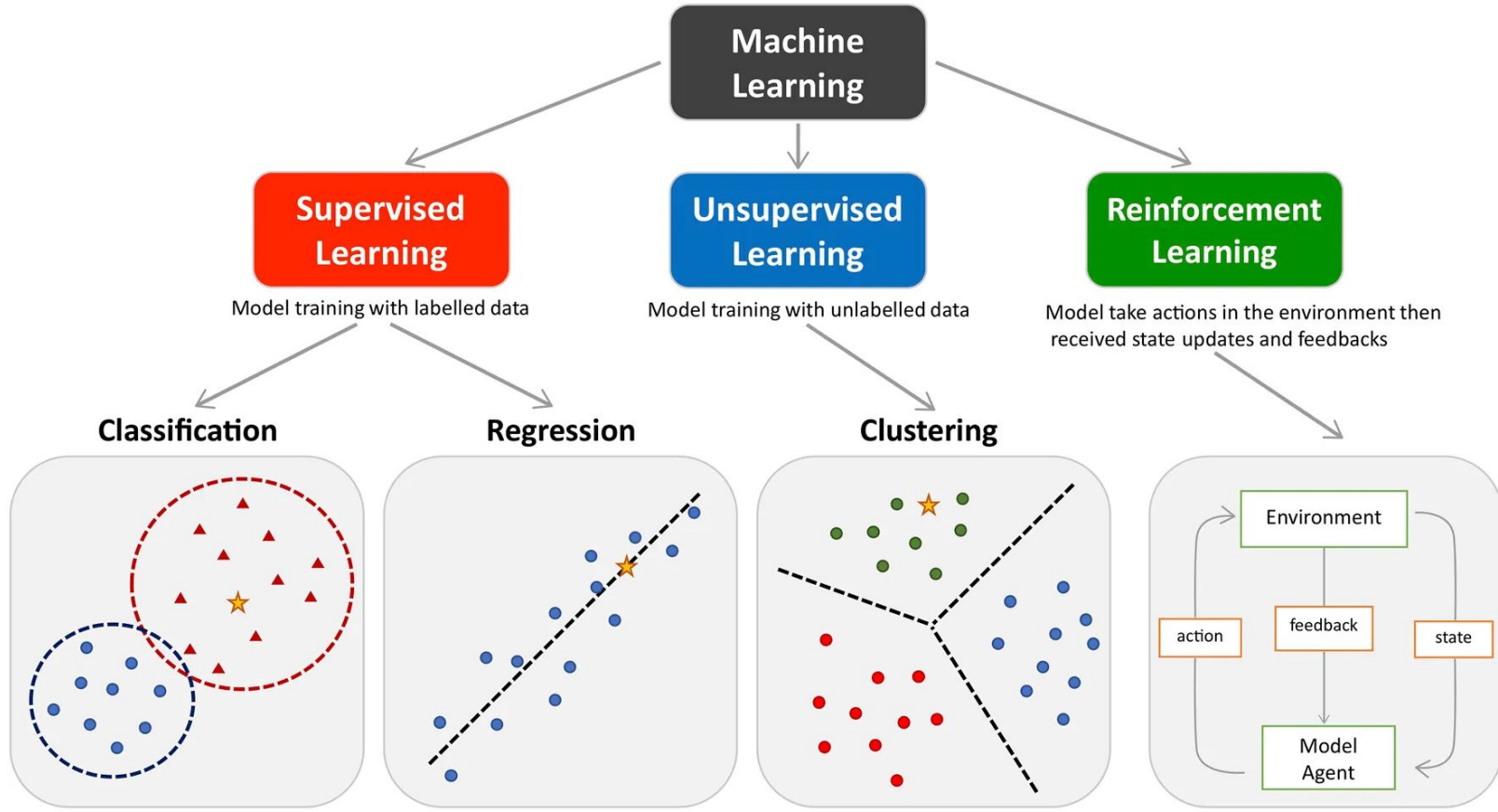
Deep Learning - II

RRC Summer School 2026

Recap

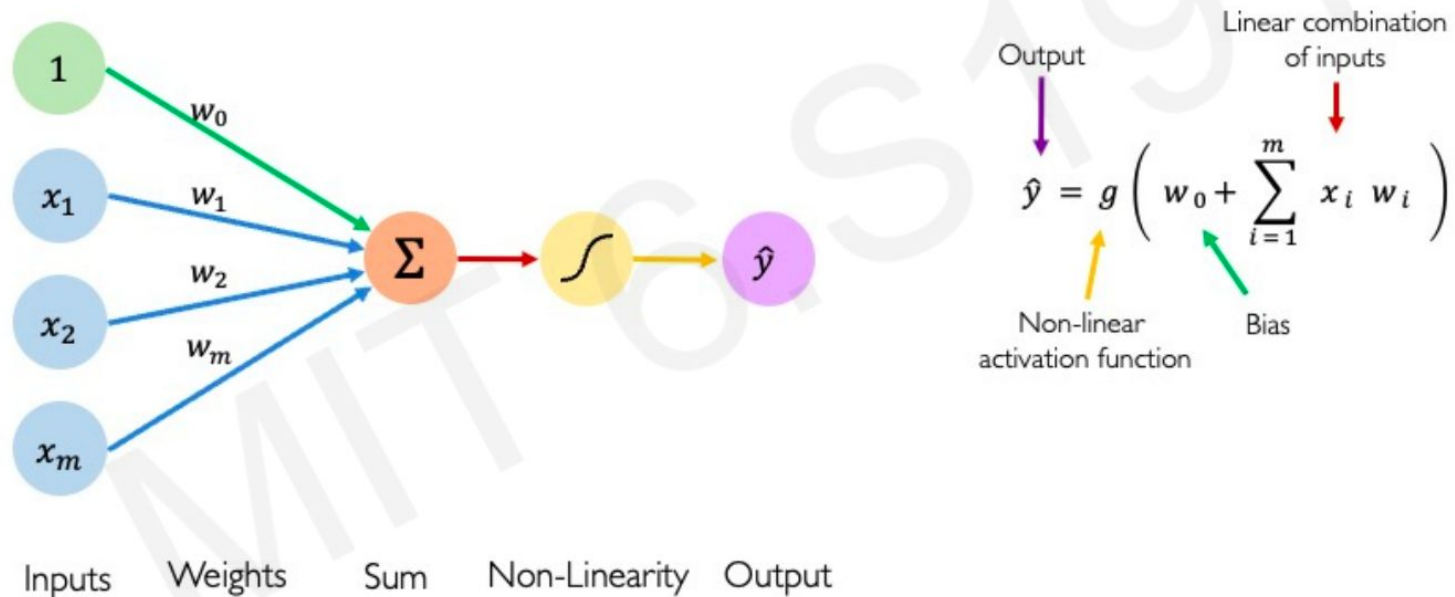
Types of Deep Learning

Classified based on availability of data and the nature of data

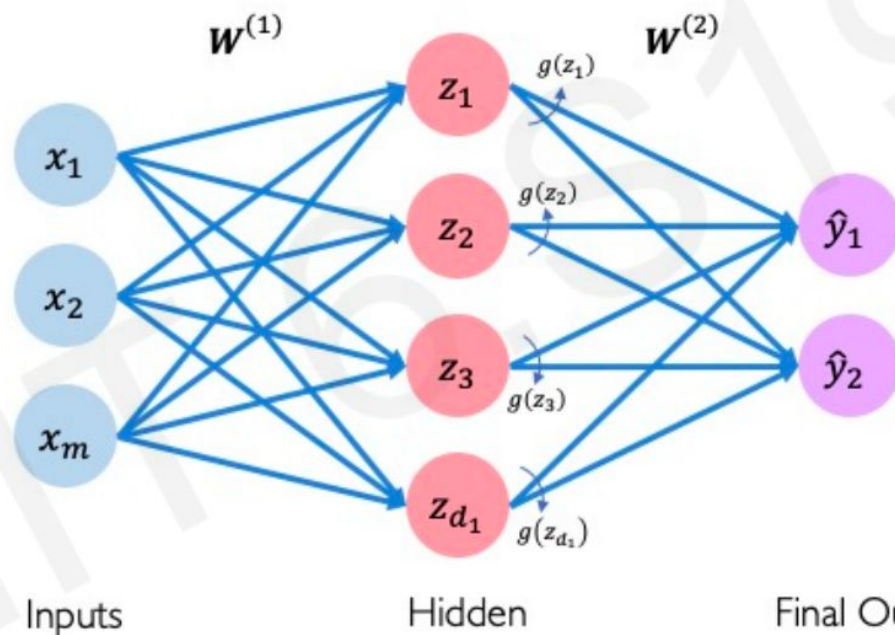


Perceptron - The structural building block of DL

The Perceptron: Forward Propagation



Single Layer Neural Network



$$z_i = w_{0,i}^{(1)} + \sum_{j=1}^m x_j w_{j,i}^{(1)} \quad \hat{y}_i = g \left(w_{0,i}^{(2)} + \sum_{j=1}^{d_1} g(z_j) w_{j,i}^{(2)} \right)$$

What is the need for activation functions?

- Hidden layer output

$$z_i = w_{0,i}^{(1)} + \sum_{j=1}^m x_j w_{j,i}^{(1)}$$

- Final output without activation

$$\hat{y}_i = w_{0,i}^{(2)} + \sum_{j=1}^{d_1} z_j w_{j,i}^{(2)}$$

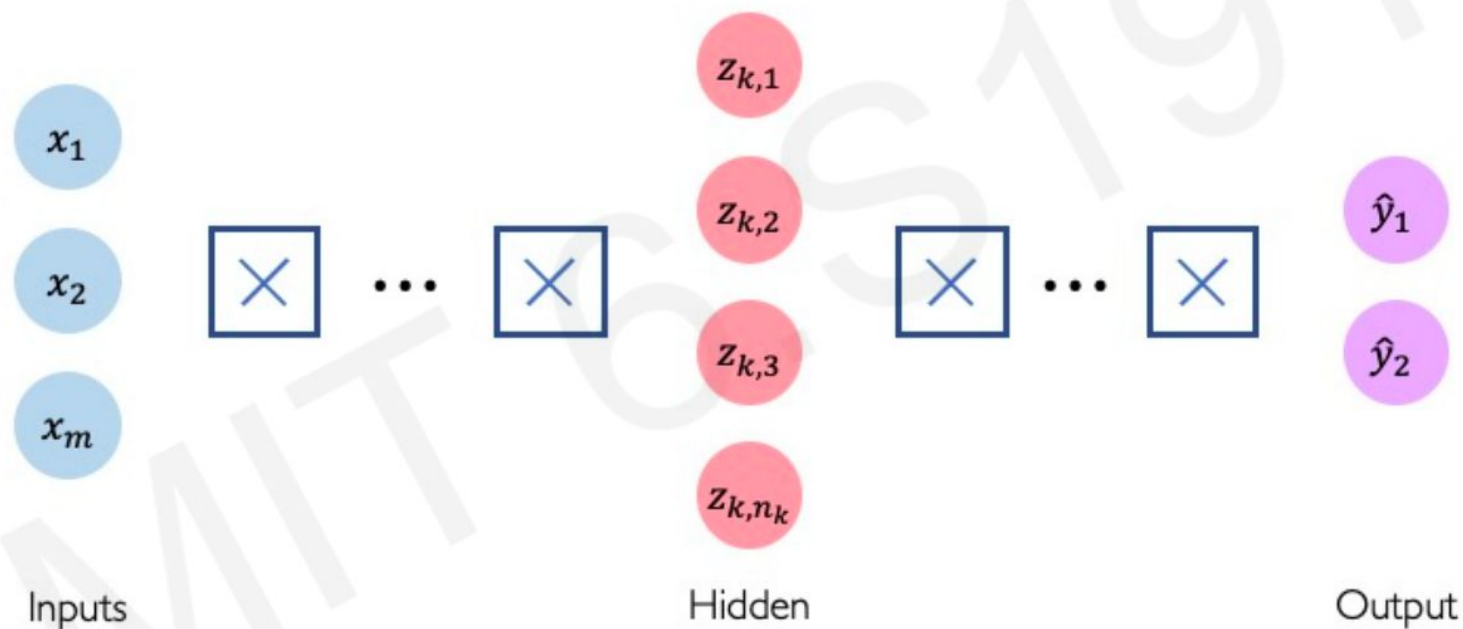
$$\hat{y}_i = w_{0,i}^{(2)} + \sum_{j=1}^{d_1} \left(w_{0,j}^{(1)} + \sum_{k=1}^m x_k w_{k,j}^{(1)} \right) w_{j,i}^{(2)}$$

$$\hat{y}_i = \left(w_{0,i}^{(2)} + \sum_{j=1}^{d_1} w_{0,j}^{(1)} \cdot w_{j,i}^{(2)} \right) + \sum_{k=1}^m x_k \left(\sum_{j=1}^{d_1} w_{k,j}^{(1)} \cdot w_{j,i}^{(2)} \right)$$

$$\hat{y}_i = w'_{0,i} + \sum_{k=1}^m x_k w'_{k,i}$$

- Deep networks can be reduced to a single layer network!
- Network cannot handle non-linear data!

Deep Neural Network



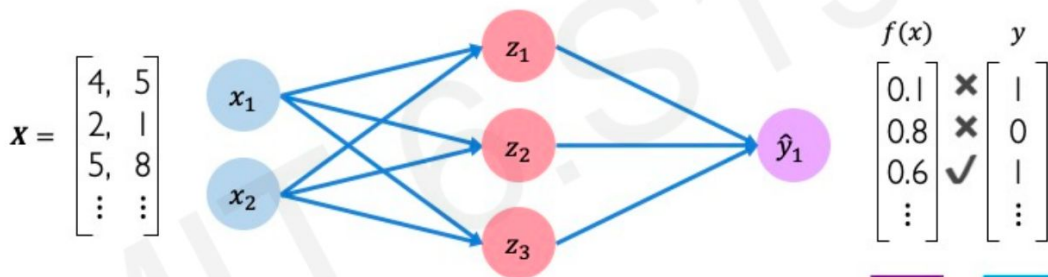
$$z_{k,i} = w_{0,i}^{(k)} + \sum_{j=1}^{n_{k-1}} g(z_{k-1,j}) w_{j,i}^{(k)}$$

Putting the model to use

- Great! We know how to construct the model. Now what?
- How do we quantify the model's performance? Loss functions!

Empirical Loss

The **empirical loss** measures the total loss over our entire dataset

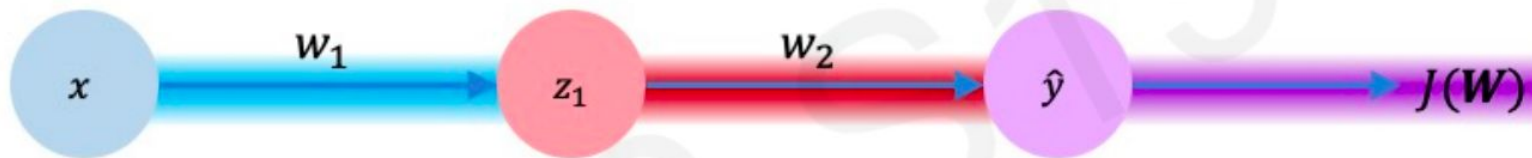


Also known as:

- Objective function
- Cost function
- Empirical Risk

$$J(W) = \frac{1}{n} \sum_{i=1}^n \mathcal{L}(\underbrace{f(x^{(i)}; W)}_{\text{Predicted}}, \underbrace{y^{(i)}}_{\text{Actual}})$$

Computing Gradients: Backpropagation



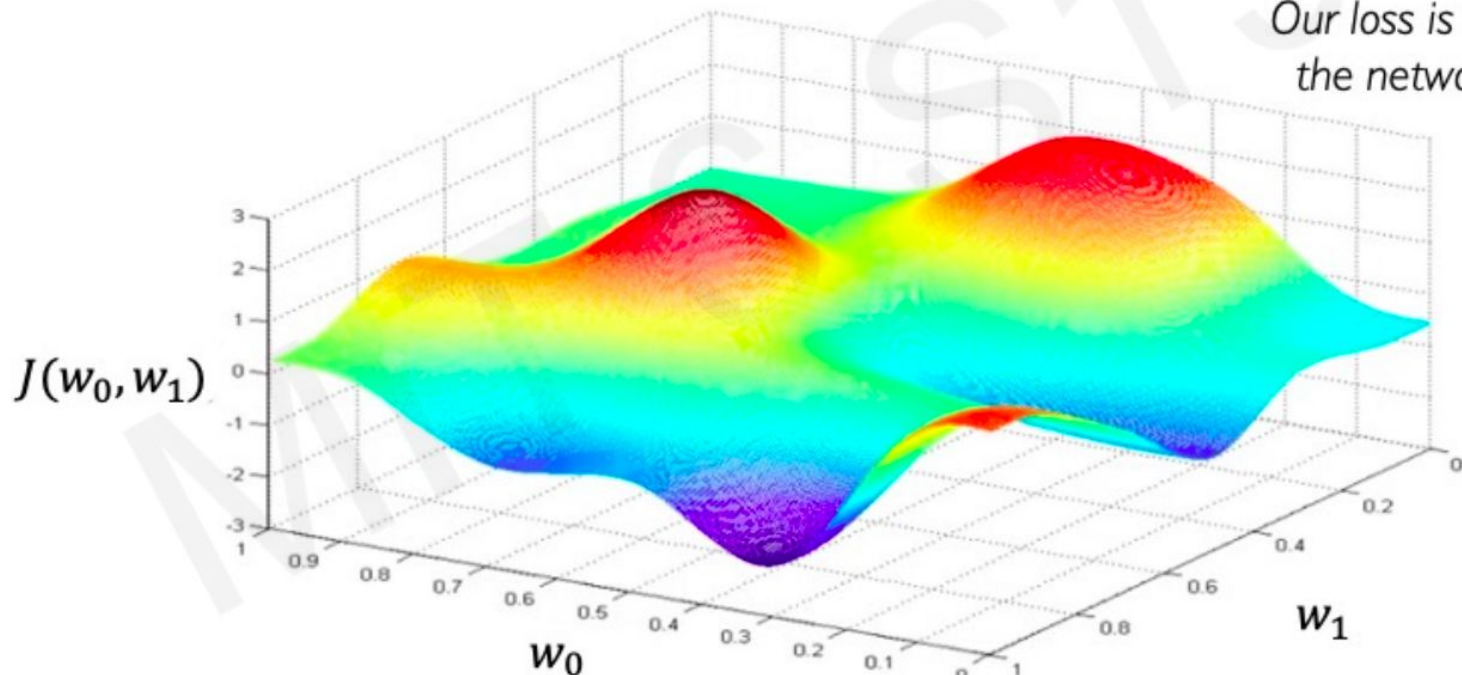
$$\frac{\partial J(W)}{\partial w_1} = \frac{\partial J(W)}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial z_1} * \frac{\partial z_1}{\partial w_1}$$

Repeat this for **every weight in the network** using gradients from later layers

Loss Optimization

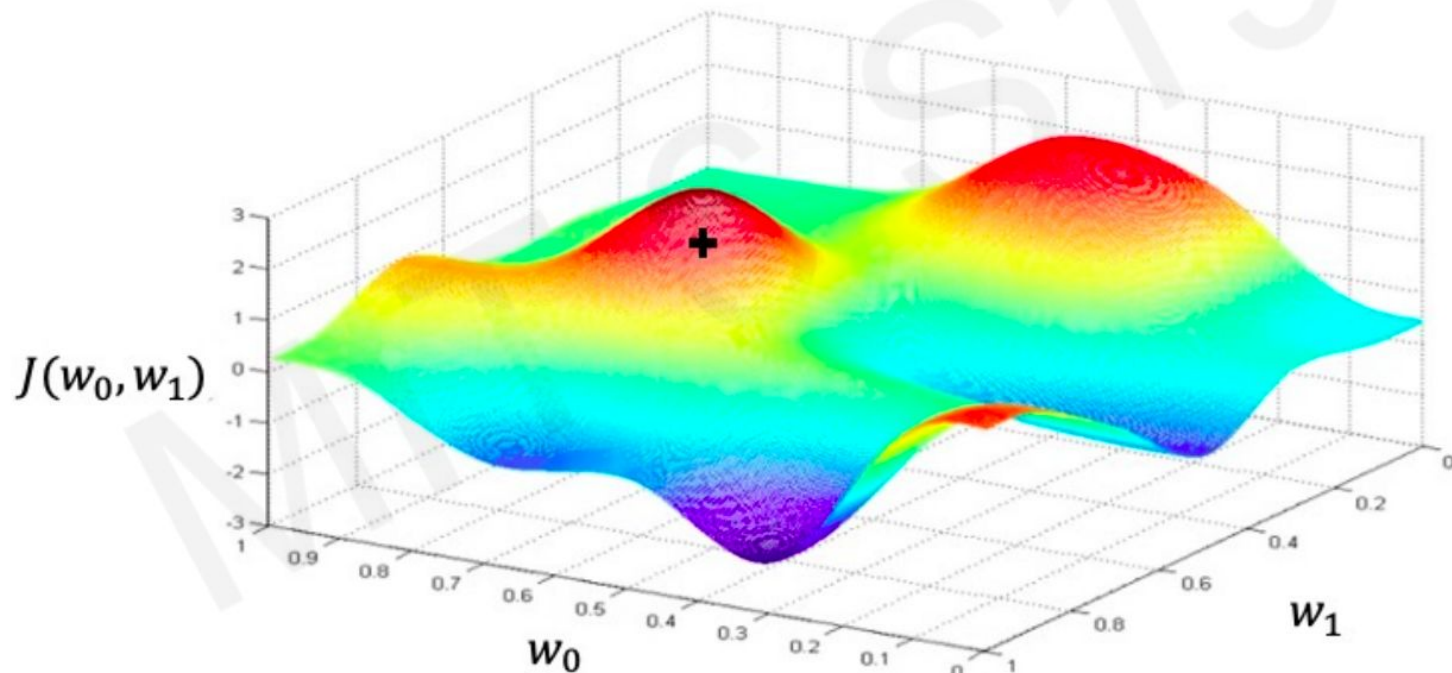
$$\mathbf{W}^* = \underset{\mathbf{W}}{\operatorname{argmin}} J(\mathbf{W})$$

Remember:
*Our loss is a function of
the network weights!*



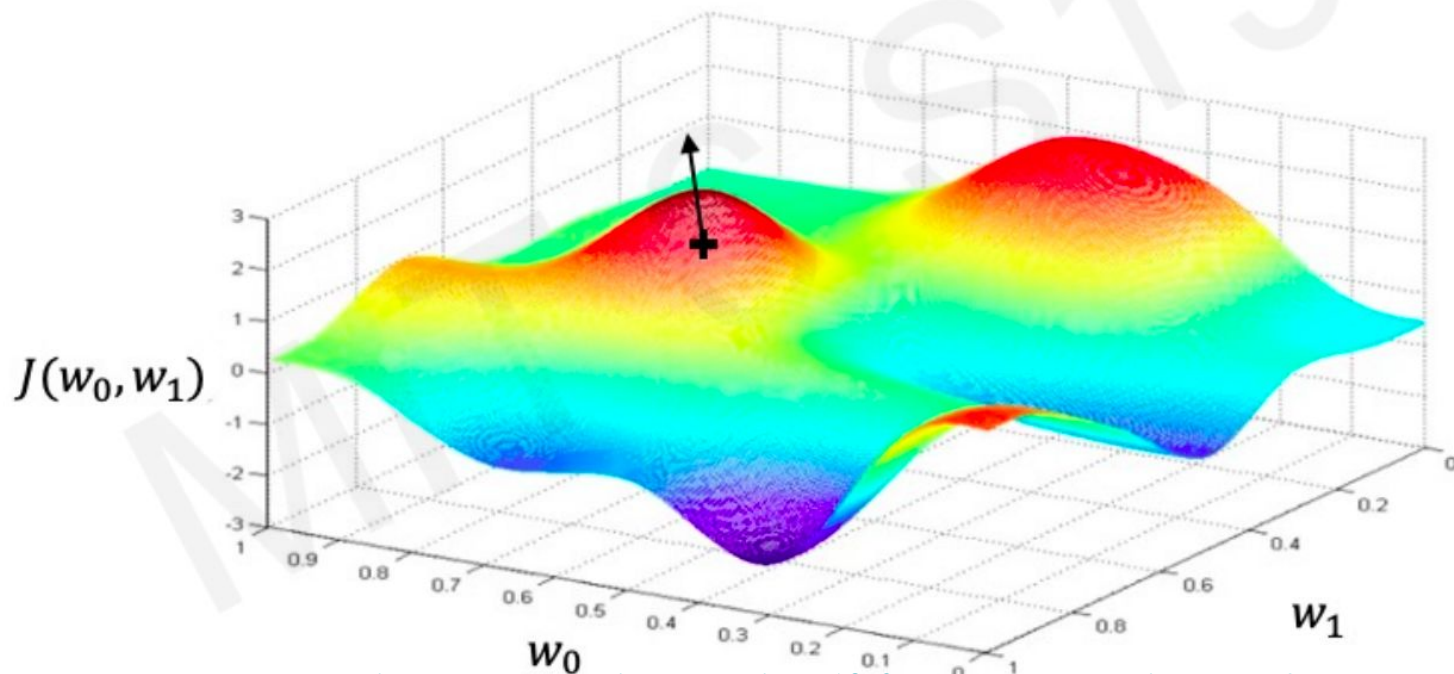
Loss Optimization

Randomly pick an initial (w_0, w_1)



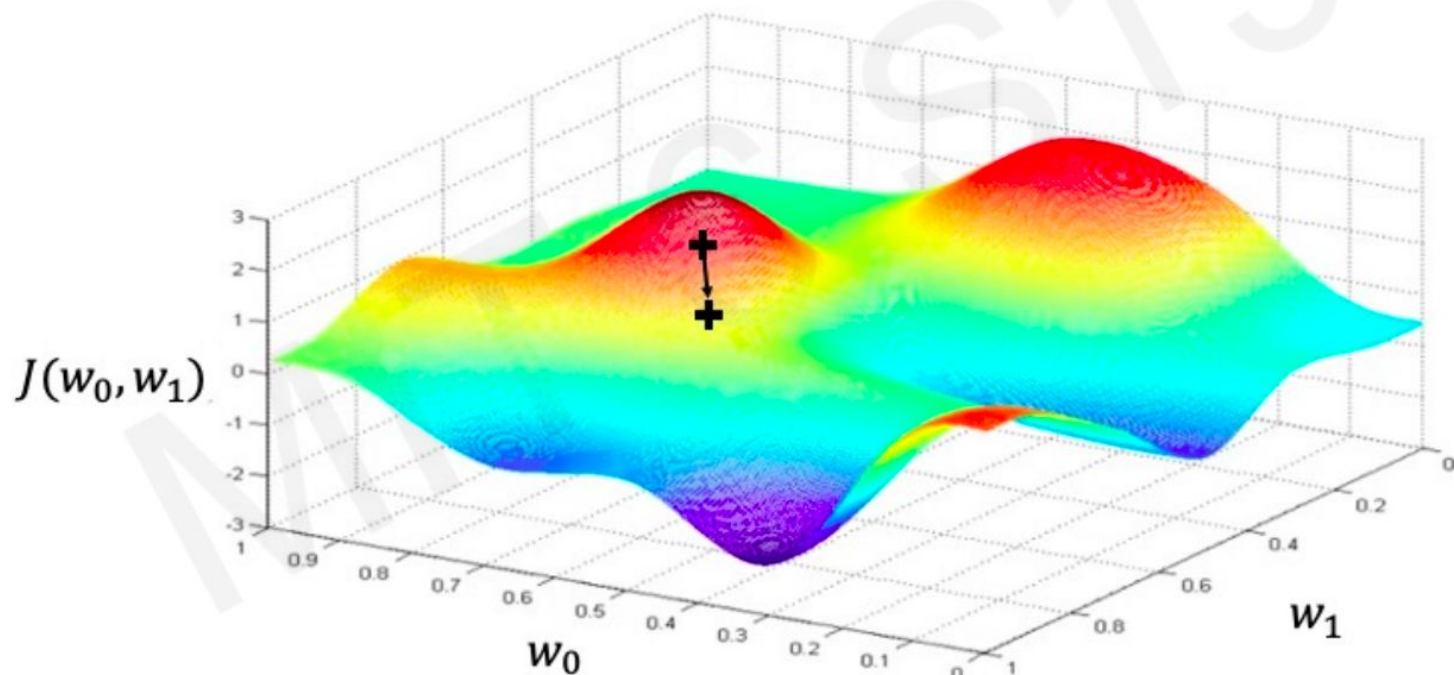
Loss Optimization

Compute gradient, $\frac{\partial J(w)}{\partial w}$



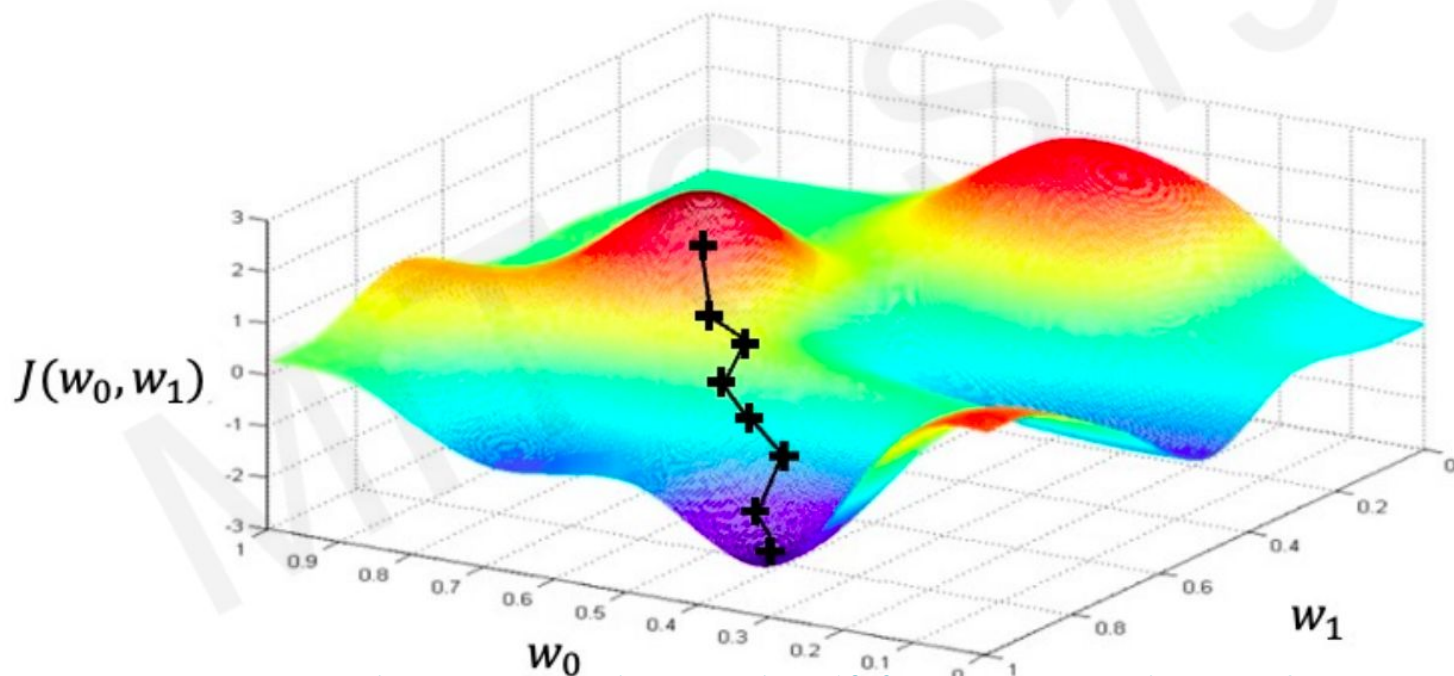
Loss Optimization

Take small step in opposite direction of gradient



Gradient Descent

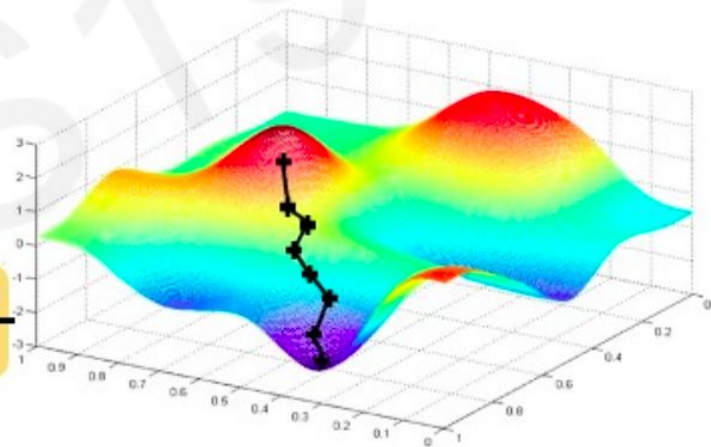
Repeat until convergence



Stochastic Gradient Descent

Algorithm

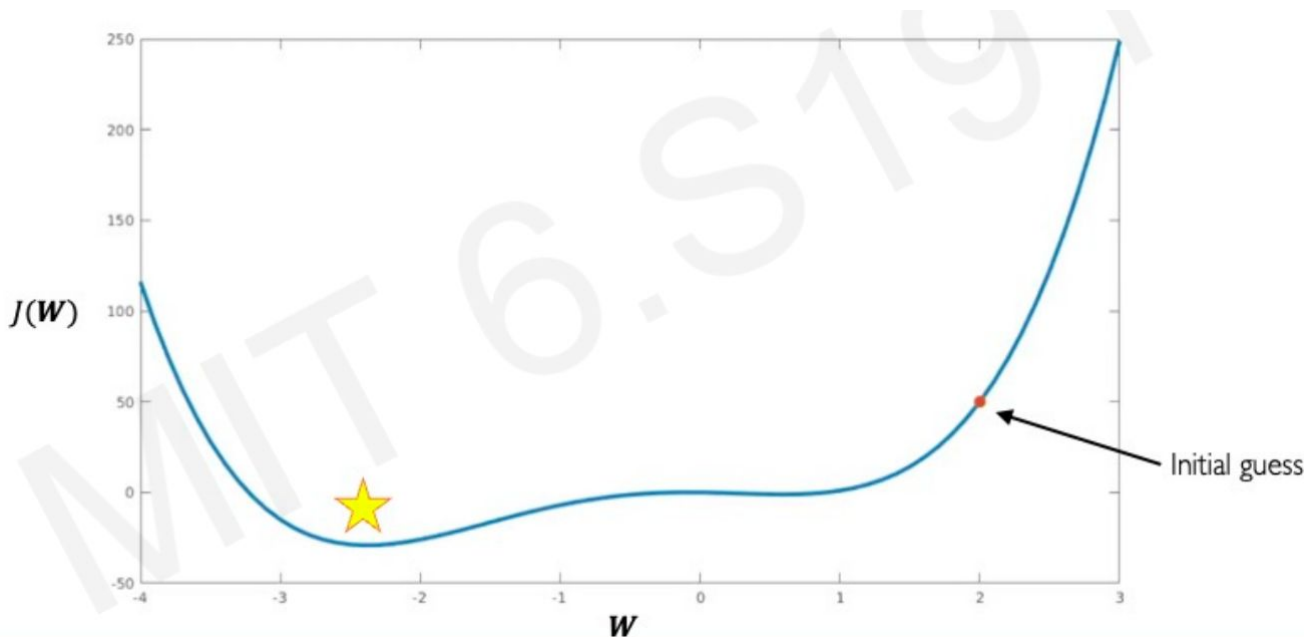
1. Initialize weights randomly $\sim \mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
3. Pick batch of B data points
4. Compute gradient, $\frac{\partial J(\mathbf{W})}{\partial \mathbf{W}} = \frac{1}{B} \sum_{k=1}^B \frac{\partial J_k(\mathbf{W})}{\partial \mathbf{W}}$
5. Update weights, $\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial J(\mathbf{W})}{\partial \mathbf{W}}$
6. Return weights



Fast to compute and a much better
estimate of the true gradient!

Setting the Learning Rate

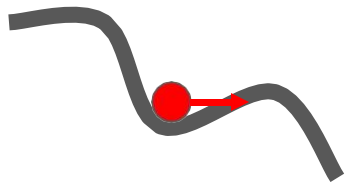
- Small learning rate converges slowly and gets stuck in false local minima
- Large learning rates overshoot, become unstable and diverge
- Stable learning rates converge smoothly and avoid local minima



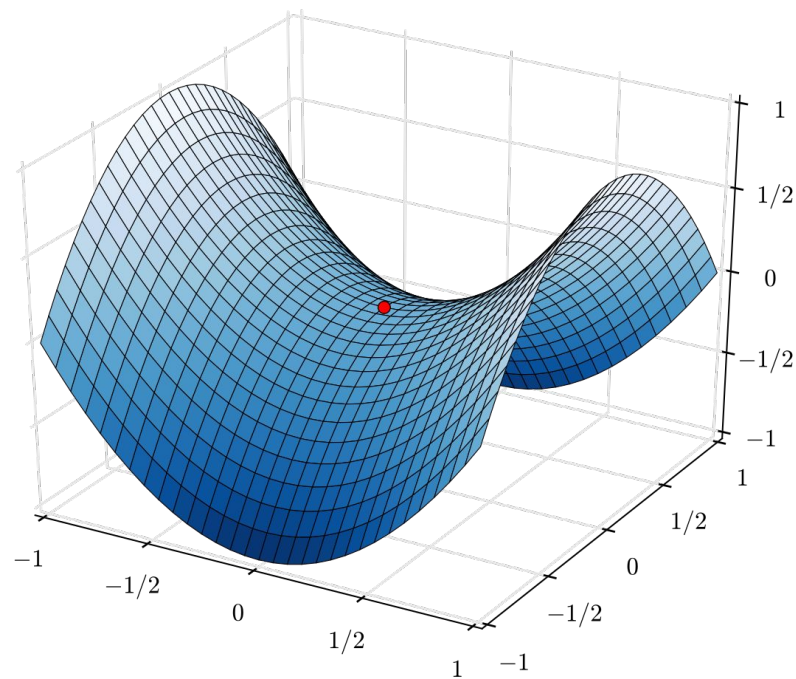
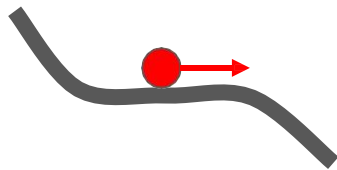
Minor Detour

Issues with SGD

Local Minima



Saddle point



SGD + Momentum

SGD

$$x_{t+1} = x_t - \alpha \nabla f(x_t)$$

```
while True:
    dx = compute_gradient(x)
    x -= learning_rate * dx
```

SGD+Momentum

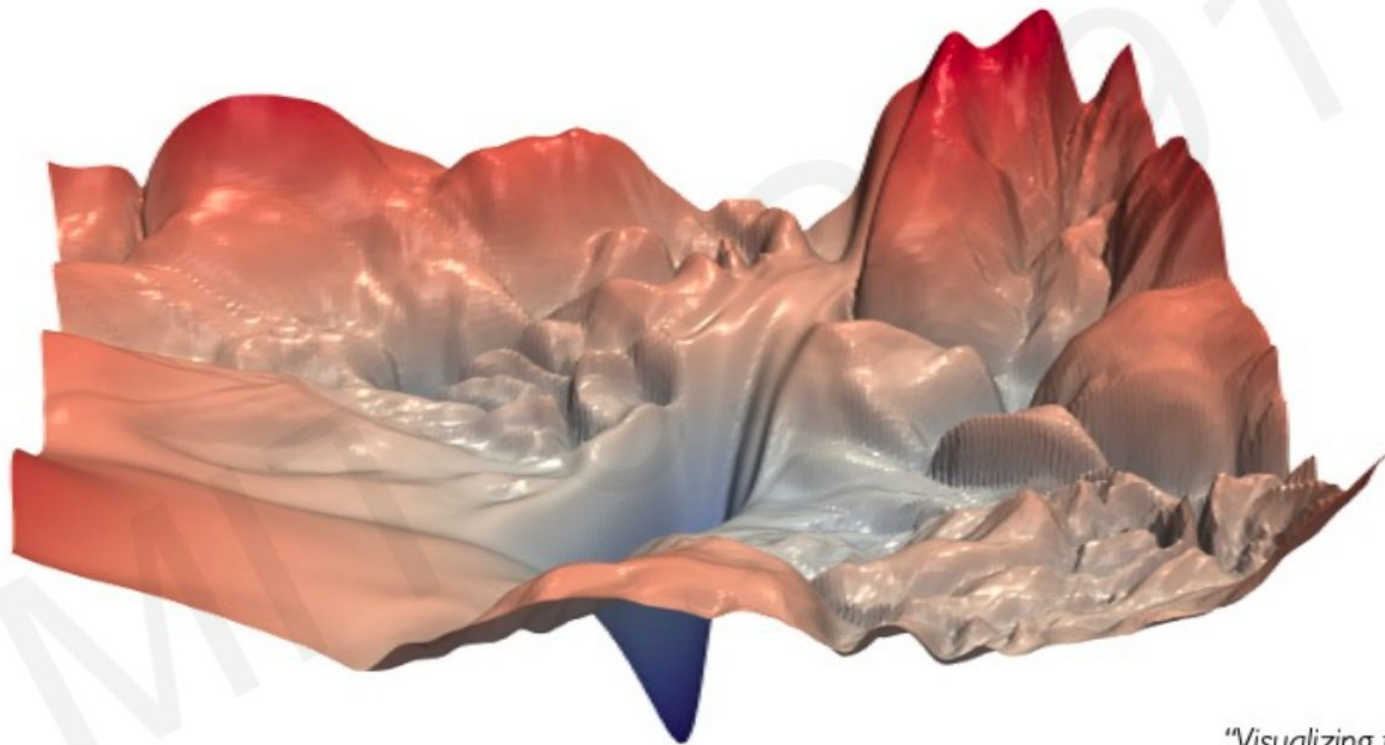
$$v_{t+1} = \rho v_t + \nabla f(x_t)$$

$$x_{t+1} = x_t - \alpha v_{t+1}$$

```
vx = 0
while True:
    dx = compute_gradient(x)
    vx = rho * vx + dx
    x -= learning_rate * vx
```

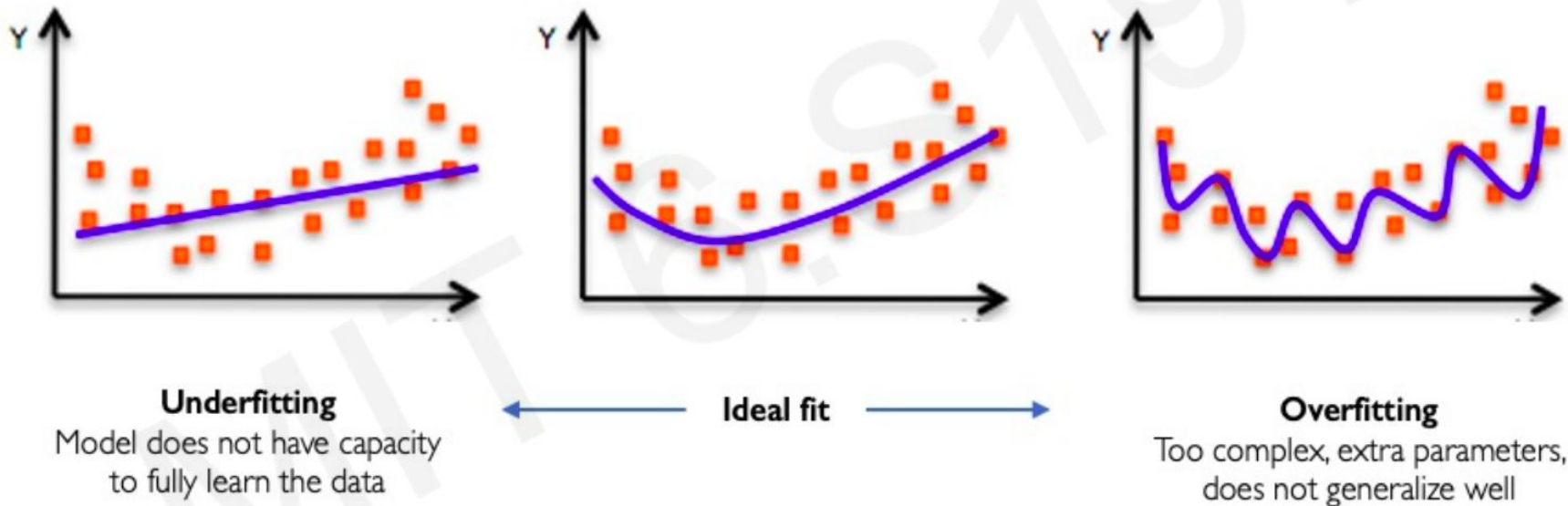
How to Train Your Model

Training Neural Networks is Difficult

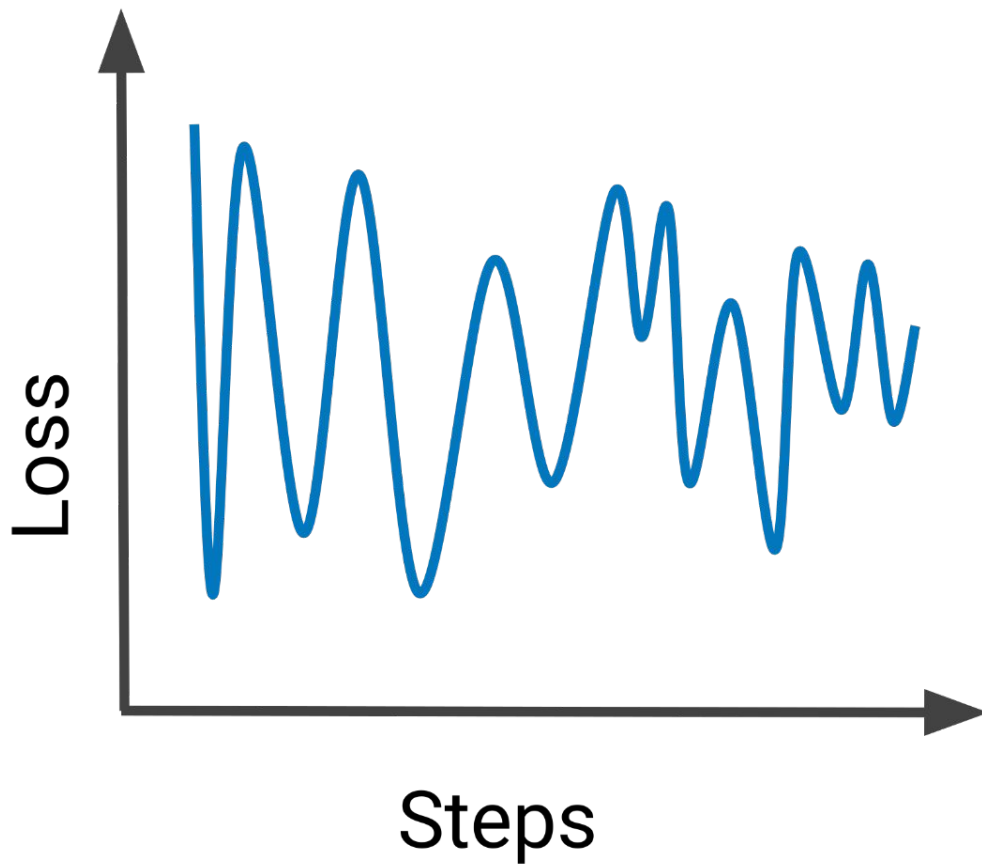


"Visualizing the loss landscape of neural nets". Dec 2017.

The Problem of Overfitting



Failure to Converge



Activation Functions (in detail)

Decision Behaviour

- Hidden layer output

$$z_i = w_{0,i}^{(1)} + \sum_{j=1}^m x_j w_{j,i}^{(1)}$$

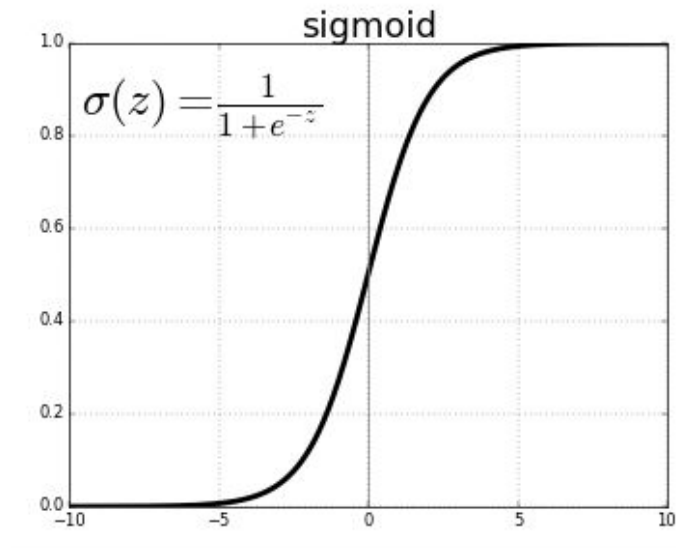
- Final output without activation

$$\hat{y}_i = w_{0,i}^{(2)} + \sum_{j=1}^{d_1} z_j w_{j,i}^{(2)}$$

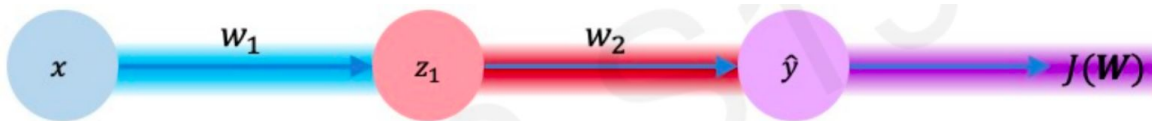
$$\hat{y}_i = w_{0,i}^{(2)} + \sum_{j=1}^{d_1} \left(w_{0,j}^{(1)} + \sum_{k=1}^m x_k w_{k,j}^{(1)} \right) w_{j,i}^{(2)}$$

$$\hat{y}_i = \left(w_{0,i}^{(2)} + \sum_{j=1}^{d_1} w_{0,j}^{(1)} \cdot w_{j,i}^{(2)} \right) + \sum_{k=1}^m x_k \left(\sum_{j=1}^{d_1} w_{k,j}^{(1)} \cdot w_{j,i}^{(2)} \right)$$

$$\hat{y}_i = w'_{0,i} + \sum_{k=1}^m x_k w'_{k,i}$$

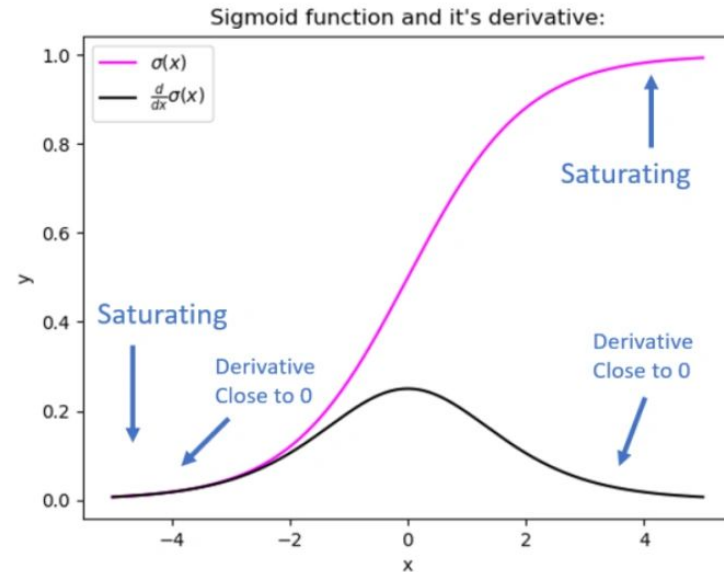


Vanishing Gradients



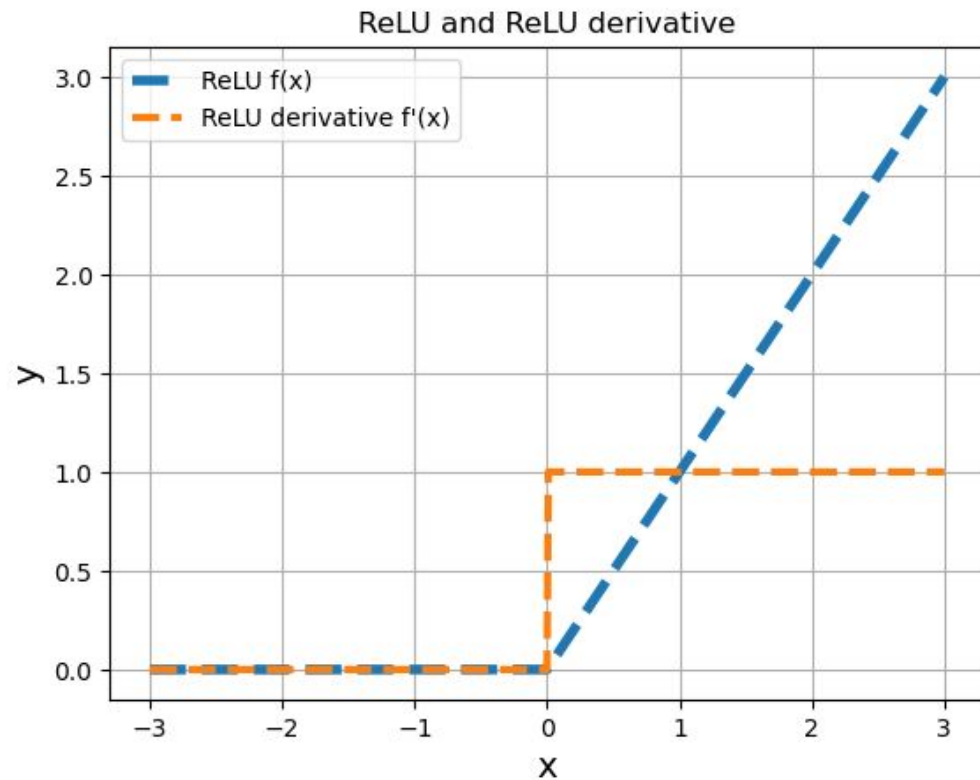
$$\frac{\partial J(W)}{\partial w_1} = \frac{\partial J(W)}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial z_1} * \frac{\partial z_1}{\partial w_1}$$

The equation is annotated with colored bars under the terms: a purple bar under $\frac{\partial J(W)}{\partial \hat{y}}$, a red bar under $\frac{\partial \hat{y}}{\partial z_1}$, and a blue bar under $\frac{\partial z_1}{\partial w_1}$.



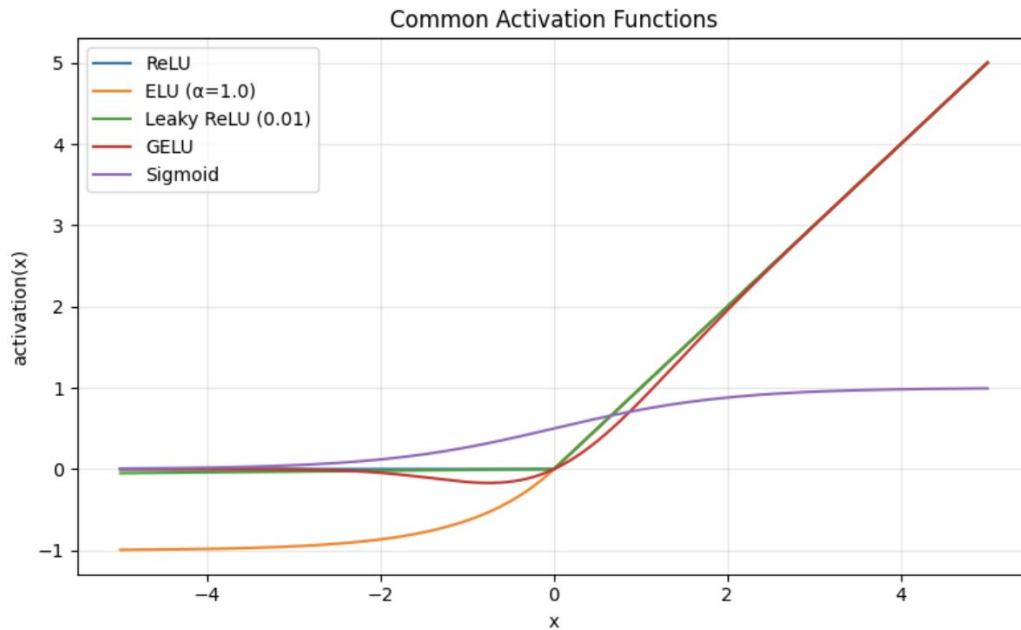
ReLU

- ReLU: $f(x) = \max(0, x)$
- Issue: Dead neurons / dying ReLU



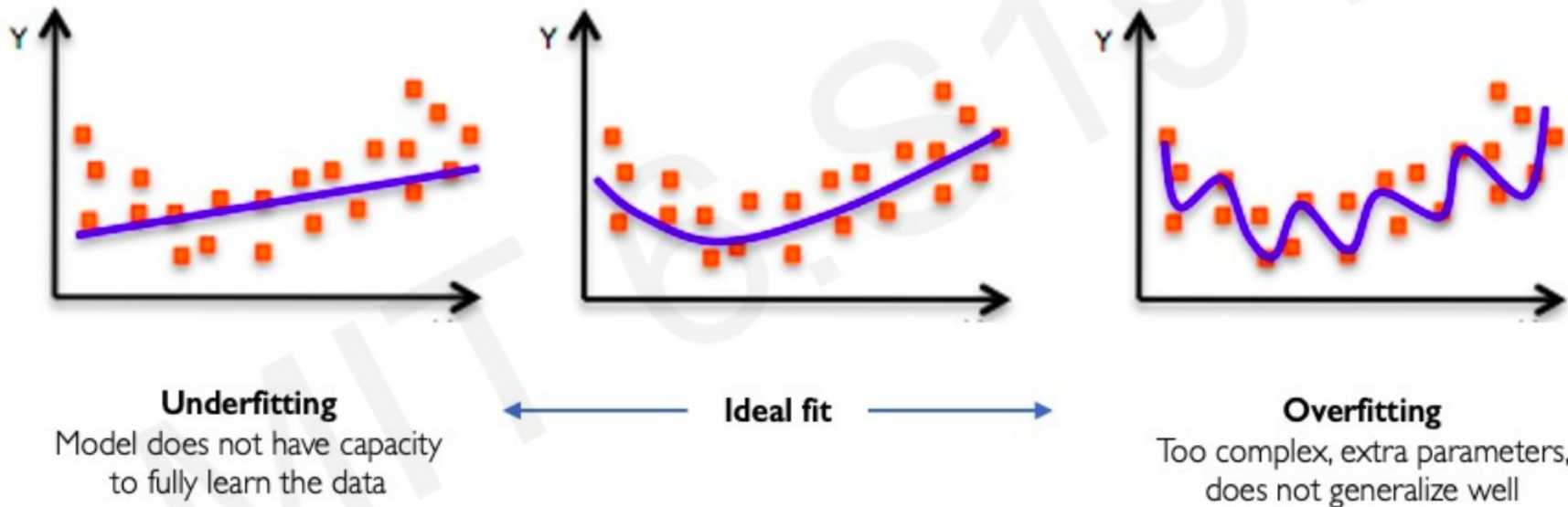
Fixes to ReLU

- Leaky ReLU
- Parametric ReLU
- ELU
- GELU
- SiLU



Regularization

The Problem of Overfitting



Regularization

$$L(W) = \frac{1}{N} \sum_{i=1}^N L_i(f(x_i, W), y_i)$$

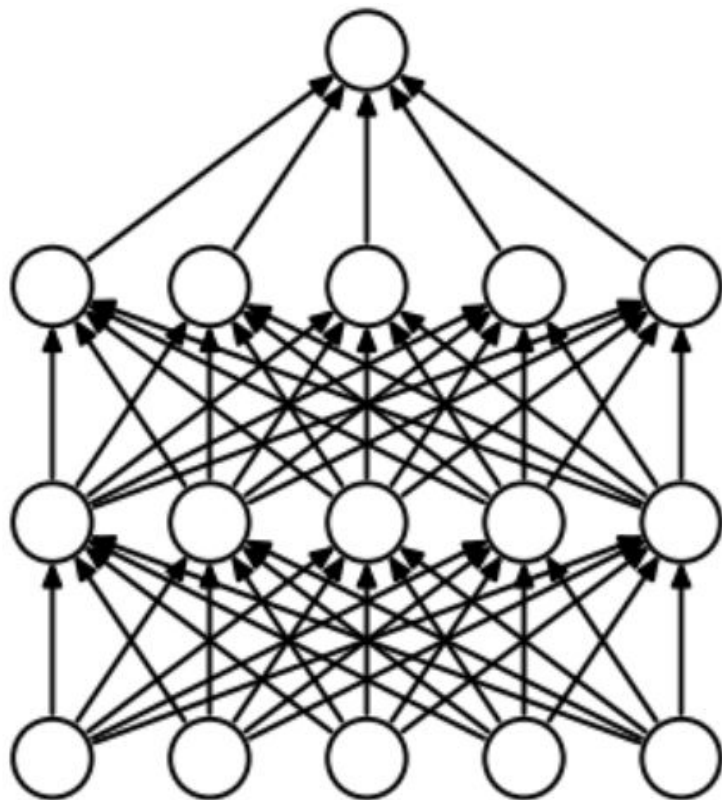
Regularization

$$L(W) = \frac{1}{N} \sum_{i=1}^N L_i(f(x_i, W), y_i) + \lambda R(W)$$

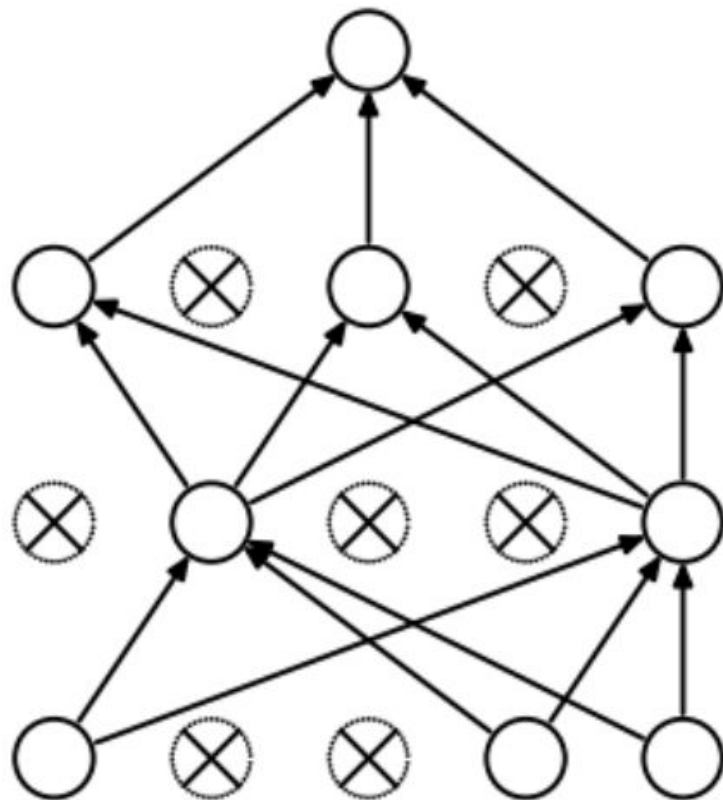
Simple examples

- L2 regularization: $R(W) = \sum_k \sum_l W_{k,l}^2$
- L1 regularization: $R(W) = \sum_k \sum_l |W_{k,l}|$

Regularization – Dropout



(a) Standard Neural Net



(b) After applying dropout.

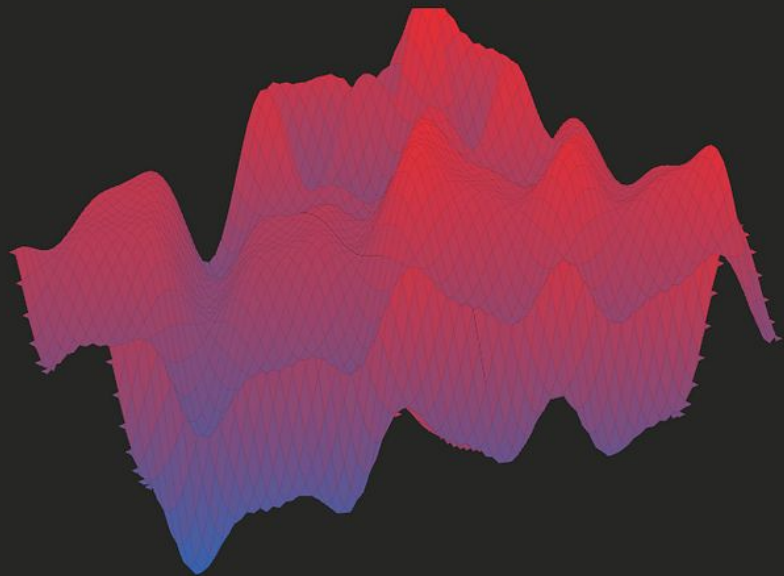
Regularization – Early Stopping



Normalization

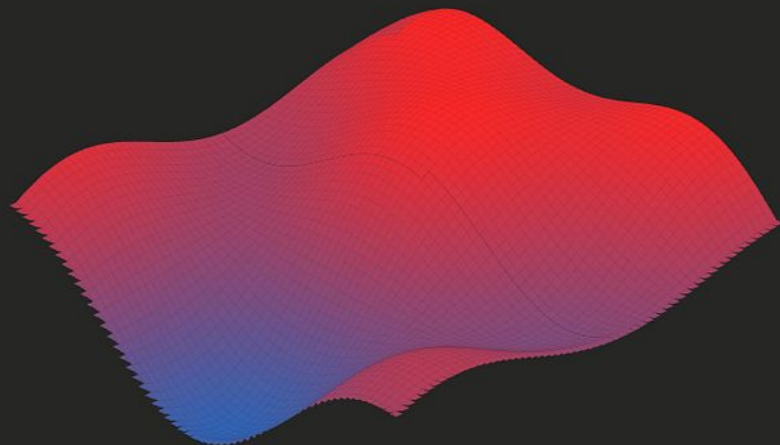
Smoothing the Landscape

Without weight decay



Sharp peaks, narrow valleys

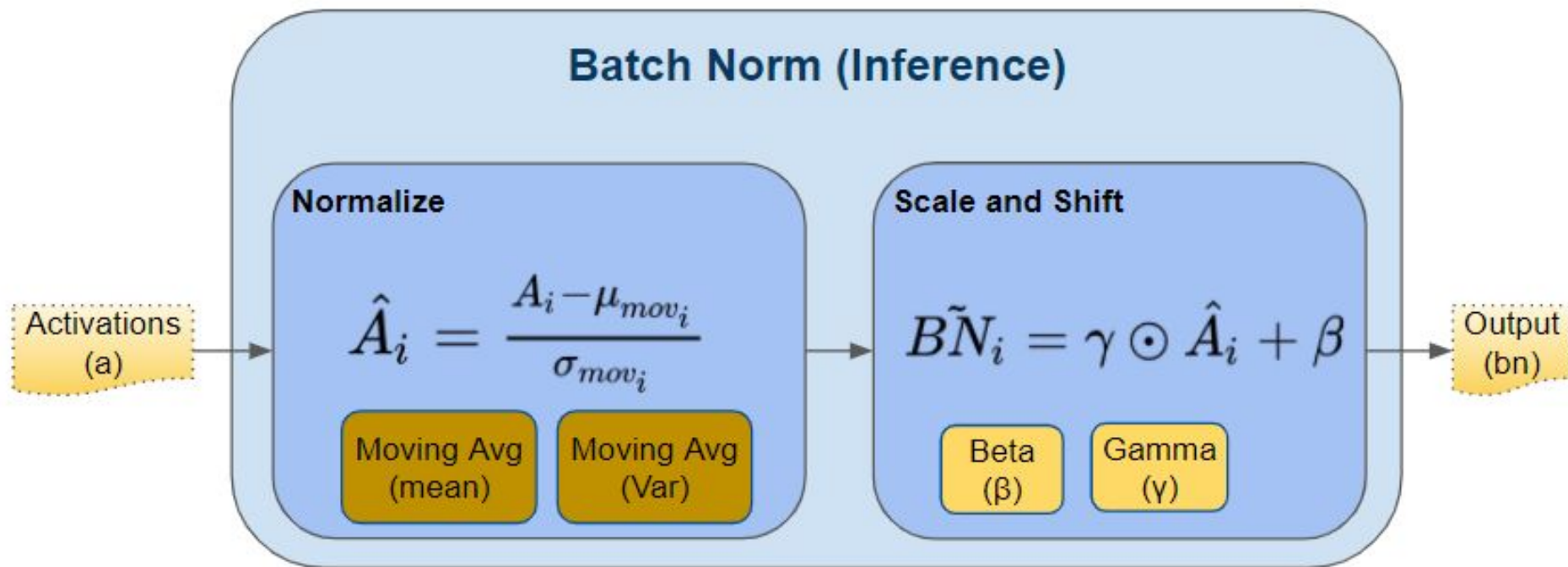
With weight decay



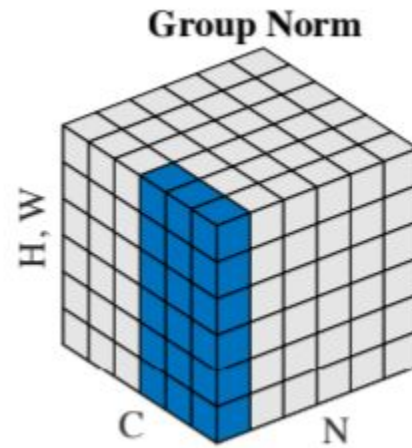
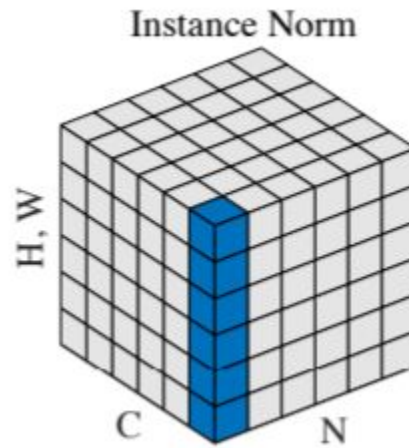
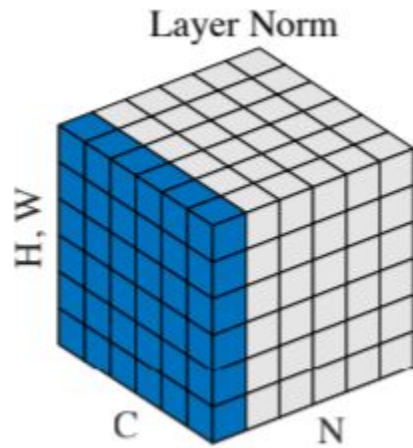
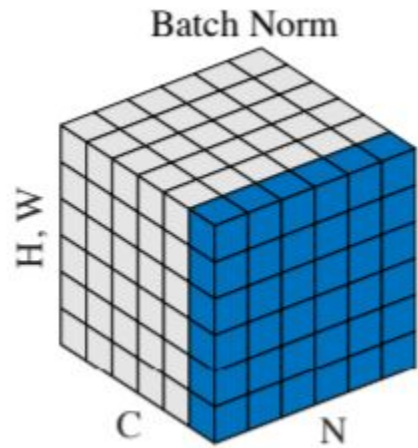
Smooth, wide valleys

— Low loss — High loss

BatchNorm



Other Norms



Data Augmentation

Augmentation

- Not just more data
- Rather about teaching the model what is important
- Improves generalization

Augmentation



(a) Original



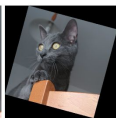
(b) Change Aspect Ratio



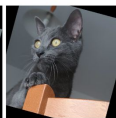
(c) Resizing



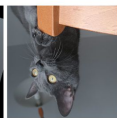
(d) Crop



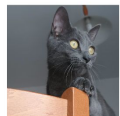
(e) Rotation + pad



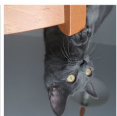
(f) Rotation



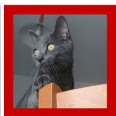
(g) Flip y



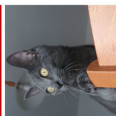
(h) Flip x



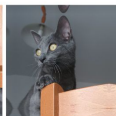
(i) Flip xy



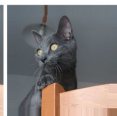
(j) Pad constant



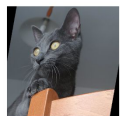
(k) Transpose



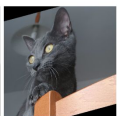
(l) Pad reflect



(m) Pad replicate



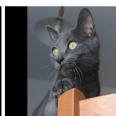
(n) Shear x



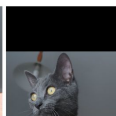
(o) Shear y



(p) Perspective



(q) Translation x



(r) Translation y



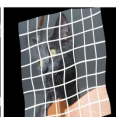
(s) Barrel



(t) Pincushion



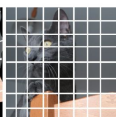
(u) Elastic



(v) Elastic, large σ



(w) Elastic, small σ



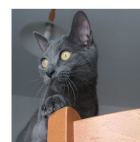
(x) Grid



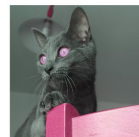
(y) Thin Plate Spline



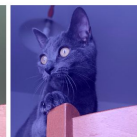
(z) Piecewise Affine



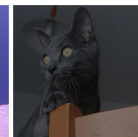
(a) Original



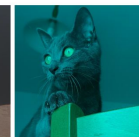
(b) Channel Shuffle



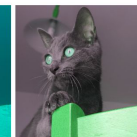
(c) Channel Shift



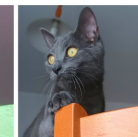
(d) Channel Multiply



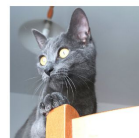
(e) Channel Dropout



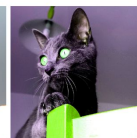
(f) Hue



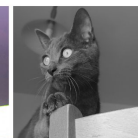
(g) Saturation



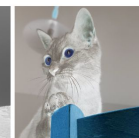
(h) Brightness



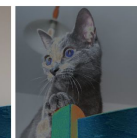
(i) Color Jitter



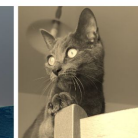
(j) Grayscale



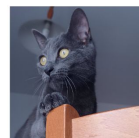
(k) Invert



(l) Solarization



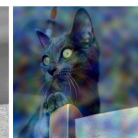
(m) Sepia



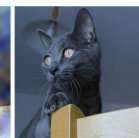
(n) FancyPCA



(o) Intensity Remap



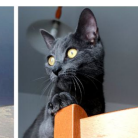
(p) FDA Simple Style Transfer*



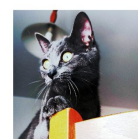
(q) Pixel Distribution Adaptation*



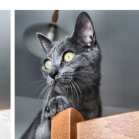
(r) Histogram Matching*



(s) Contrast



(t) Hist Equalization



(u) CLAHE

Augmentation



(a) Original



(b) AverageBlur

(c) MedianBlur

(d) GaussianBlur

(e) Generalized Normal Filter

(f) BilateralBlur

(g) JointBilateralBlur



(h) Defocus

(i) GlassBlur

(j) ZoomBlur

(k) MotionBlur

(l) UnsharpMask

(m) Emboss



(n) Dilation

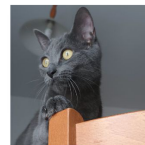
(o) Erosion

(p) Opening

(q) Closing

(r) Posterize

(s) Cartoon



(a) Original



(b) Salt

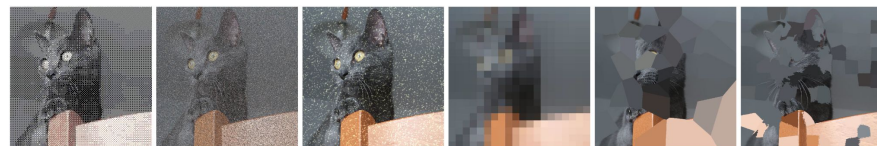
(c) Pepper

(d) Gaussian

(e) Laplace

(f) Poisson

(g) ISO



(h) Dithering

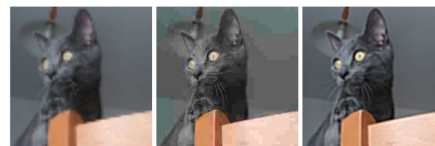
(i) ShufflePixels

(j) Spatter

(k) Pixelization

(l) Voronoi

(m) Superpixels



(n) Downscaling

(o) Compression

(p) Ringing